

How to Learn Java for DSA — Complete Roadmap

Part 1 — Java Basics

1. What to learn (the essential subset)
2. What to skip completely
3. How long this should take

Part 2 — Core DSA Topics (in order)

4. Arrays & Strings
5. Recursion & Backtracking
6. Linked Lists, Stacks & Queues
7. Trees & Graphs
8. Sorting & Searching
9. Dynamic Programming
10. Heaps, HashMaps & Sets

Part 1 — Java Basics

1. What to learn →

- **int, long, double, char, boolean** — these are your basic data types. Use **long** when numbers get very large, **int** for most cases.
- **if/else, for loop, while loop** - used in every single DSA problem. Get these solid first.
- **Methods** - how to write a function, pass values into it, and return a result. Critical later when you hit recursion.
- **Arrays** - 1D arrays for most problems, 2D arrays for grid/matrix problems. Learn how to traverse and update them.
- **String and StringBuilder** — String for reading, StringBuilder for building. Never use String + String inside a loop in Java — it is very slow. Always use StringBuilder there.
- **Collections — this is the most important part:**
 - **ArrayList** - a dynamic array, size grows automatically
 - **HashMap** - stores key-value pairs, like a dictionary

- **HashSet** - stores only unique values, great for checking duplicates
- **Stack** - last in first out, used in many tree and string problems
- **Queue / ArrayDeque** - first in first out, used heavily in BFS.
- **PriorityQueue** - gives you the smallest or largest element instantly

2. What to skip

- **OOP** - inheritance, interfaces, abstract classes. None of this is needed to solve DSA problems.
- **Generics** - you will see `ArrayList<Integer>` everywhere. Just use it as is. Don't go deep into how generics work.
- **Streams and Lambdas** - great for real projects, completely useless for DSA right now.
- **Exception handling** - try/catch blocks. Not needed for coding problems.
- **File I/O** - reading and writing files. Never comes up in DSA.

- **Threads** - nothing to do with DSA. Skip entirely.
- **Frameworks** - Spring Boot, Maven, Hibernate are for building applications. Not for solving problems.

3. How long

- **2–3 weeks max** - more than this means you're going too slow.
- **Day 1–4** → syntax, loops, methods. Write factorial, patterns, even/odd.
- **Day 5–8** → arrays & strings. Reverse array, find max, palindrome check.
- **Day 9–14** → collections. ArrayList, HashMap, HashSet — just the basic methods.
- **After 2 weeks** → **start DSA**. Don't wait to feel ready. Just start.

Part 2 — Core DSA Topics

1. Arrays & Strings

- **Array** - a list of elements stored in order. Like boxes in a row, each with a number.
- **What to learn:**
 - Loop through an array
 - Find max, min, sum
 - Reverse an array
 - Find if a number exists
- **Two pointers** - take one pointer at start, one at end. Move them toward each other. Used when you need to find a pair or check something from both sides.
- **Sliding window** - take a fixed window of elements, slide it one step at a time. Used when the question says "subarray of size K" or "longest substring."
- **Prefix sum** - before solving, compute the running sum of the array. Then any range sum is just one subtraction. Saves a lot of time.
- **Strings** - just an array of characters. Most array tricks work on strings too.
- **StringBuilder** — when you need to build a string step by step, use StringBuilder. Not plain String.

Plain String is slow in Java.

- **Common problems to practice:**

- Reverse an array
- Find max in subarray
- Two sum
- Longest substring without repeating characters
- Check if string is palindrome

- **How many** - 25 to 30 problems. Don't rush.
Understand the pattern behind each solution.

2. Recursion & Backtracking

- **Recursion** - a function that calls itself. Breaks a big problem into a smaller version of the same problem.
- **Two parts every recursive function needs:**
 - Base case - when to stop
 - Recursive case - call yourself with a smaller input
- **Simple example** - factorial of 5 is $5 \times$ factorial of 4. Factorial of 1 is just 1. That's your base case.
- **Call stack** - every recursive call waits in memory until the smaller call finishes. Draw this on paper. It will make recursion click.

- **How to think** - assume your function already works for smaller input. Just figure out how to use that result for the current input. This is the key mindset.
- **Backtracking** - recursion with an undo step. Make a choice, go deep, if it doesn't work undo and try next option.
- **Common problems to practice:**
 - Factorial
 - Fibonacci
 - Reverse a string using recursion
 - Generate all subsets of an array
 - Generate all permutations
 - N-Queens
- **How many** - 15 to 20 problems. Don't rush this topic. Everything after this — trees, graphs, DP — needs recursion to be solid.

3. Linked Lists, Stacks & Queues

- **Linked List** - a chain of nodes. Each node has a value and a pointer to the next node. No index like arrays — you have to walk from the start.

- **What to learn:**

- Traverse a linked list
- Insert and delete a node
- Reverse a linked list
- Find the middle node
- Detect a cycle - use slow and fast pointer (Floyd's algorithm)

- **Stack** - last in first out. Like a stack of plates — you add and remove from the top only.

- Use Java's `Stack` class or `ArrayDeque`
- Used for — valid parentheses, next greater element, undo operations

- **Queue** - first in first out. Like a line of people — first one in is first one out.

- Use `ArrayDeque` in Java
- Used heavily in BFS later

- **Monotonic stack** - a stack where elements are always in increasing or decreasing order. Sounds hard but very useful in interview problems.

- **Common problems to practice:**

- Reverse a linked list
- Find middle of linked list
- Valid parentheses
- Implement queue using two stacks

- Next greater element

- **How many** — 20 to 25 problems.

4. Trees & Graphs

- **Tree** - a connected structure with nodes. Each node can have children. No cycles.
- **Binary tree** - each node has at most 2 children — left and right.
- **4 traversals you must know:**
 - Inorder — left, root, right
 - Preorder — root, left, right
 - Postorder — left, right, root
 - Level order — level by level using a queue (BFS)
- **BST (Binary Search Tree)** - left child is always smaller, right is always larger. Inorder of BST gives sorted output.
- **What to learn in trees:**
 - Height and depth
 - Diameter
 - Lowest common ancestor
 - Check if balanced
 - Check if BST

- **Graph** - nodes connected by edges. Can be directed or undirected.
- **How to store a graph in Java** - adjacency list. Use `HashMap` or array of `ArrayList`.
- **BFS** — level by level traversal using a queue. Used for shortest path.
- **DFS** — go as deep as possible using recursion or stack. Used for cycle detection, connected components.
- **Common problems to practice:**
 - Max depth of binary tree
 - Level order traversal
 - Lowest common ancestor
 - Number of islands
 - Detect cycle in graph
- **How many** — 30 to 35 problems. This is the most important section.

5. Sorting & Searching

- **Sorting** — arranging elements in order. You need to implement at least two from scratch.

- **Merge sort** — divide array in half, sort each half, merge them back. Always $O(n \log n)$. Stable sort.
- **Quick sort** — pick a pivot, put smaller elements on left, larger on right, repeat. $O(n \log n)$ average but $O(n^2)$ worst case.
- **Don't just use `Arrays.sort()`** - know how merge sort and quick sort work internally. Interviews ask this.
- **Binary search** - only works on sorted arrays. Cut the search space in half every step. $O(\log n)$.
- **Binary search is a pattern, not just a function:**
 - Search in rotated sorted array
 - Find first and last position
 - Binary search on the answer — when the question asks for minimum or maximum value that satisfies a condition
- **Common problems to practice:**
 - Binary search basic
 - Search in rotated sorted array
 - Find peak element
 - Koko eating bananas
 - Implement merge sort

- **How many** — 15 to 20 problems.

6. Dynamic Programming

- **DP** - solve a big problem by breaking it into smaller subproblems. Store the results so you don't compute the same thing twice.
- **Two ways to do DP:**
 - **Memoization (top-down)** - write a recursive solution, add a cache to store results. Start here.
 - **Tabulation (bottom-up)** - build the answer from smallest subproblem upward using a table. Learn this after memoization clicks.
- **Always start with memoization.** It is easier to think about and closer to the recursive solution.
- **5 patterns you must know:**
 - **0-1 Knapsack** — include or exclude each item
 - **Longest Common Subsequence** — 2D DP on two strings
 - **Longest Increasing Subsequence** — 1D DP on one array
 - **Coin Change** — minimum coins to make a target amount

- **House Robber** — can't pick adjacent elements, maximize sum
- **Common problems to practice:**
 - Climbing stairs
 - House robber
 - Coin change
 - Longest common subsequence
 - 0-1 Knapsack
- **How many** — 25 to 30 problems. Give this at least 4 weeks. It is the hardest topic. That is normal.

7. Heaps, HashMaps & Sets

- **Heap** - a special structure that always gives you the min or max element instantly.
 - In Java use `PriorityQueue`
 - Default is min-heap — smallest element comes out first
 - For max-heap — pass `Collections.reverseOrder()`
- **When to use heap** - any problem that says "top K elements", "K largest", "K closest", "median of stream."
- **HashMap** - stores key-value pairs. Get and put are $O(1)$.
 - Use for frequency counting, caching, grouping

- Most used line in DSA — `map.put(key, map.getOrDefault(key, 0) + 1).`
- **HashSet** - stores only unique values. Contains is $O(1)$.
 - Use for checking duplicates, visited nodes, dictionary lookup
- **Common problems to practice:**
 - Top K frequent elements
 - Kth largest element
 - Two sum using HashMap
 - Longest consecutive sequence using HashSet
 - Find median from data stream
- **How many — 15 to 20 problems.**